SRS Document:

Restaurant App MVP

Software Requirements Specification (SRS)

- **INTRODUCTION**

1.1: Purpose:

This document describes the requirements for the Restaurant App MVP.

The app allows customers to browse the menu, search items, reserve a table,

and place an order, while restaurant managers can manage the menu, view

orders, and handle reservations. This version (Assignment 2) extends the app with a Node.js and Express server and a MongoDB database. All devices read and write the same data through REST APIs, so changes made by a manager are visible to every customer. This document is intended

for the developer building the app and for evaluation purposes.

1.2: Scope:

- ***In Scope***:

- User signup and login (Customer and Manager roles).

- Browsing the menu with categories and search.

- Viewing item details and daily specials.

- Adding items to cart and applying promo codes.

- Placing an order (Dine-in or Takeaway).

- Reserving a table.

- Tracking order status.

- Manager dashboard to manage menu, orders, and reservations.

- ***Out of Scope:***

- Real online payments.

- Push notifications.

- Backend server or database.

- Real-time GPS delivery tracking.

1.3: Definitions & Acronyms:

- ***SRS*** : Software Requirements Specification, the document describing what the system should do.

- ***MVP*** : Minimum Viable Product, the basic working version of the app with only core features.

- ***UML*** - Unified Modeling Language, used to draw diagrams that show how the system works.

- ***Hook*** : A special function in React (like useState) that lets a component use features such as state or lifecycle events.

- ***Context*** : A React feature used to share data (like user login info) across many screens without passing it manually.

- ***Reducer*** : A function that decides how the app's state should change based on a given action.

- ***Mock Data saved in app***: Data stored in MongoDB through the server.

- ***FlatList*** :A React Native component used to display a scrollable list of items, such as the menu.

➢ **OVERALL DESCRIPTION:**

2.1: User Roles:

This app has two types of users :

- Customer
- Restaurant manager

2.2: User Stories:

1. As a customer, I want to see the menu on my phone, so that I can order easily.

2. As a customer, I want to book a table early, so that I don't have to wait.

3. As a customer, I want to see today's special items, so that I can pick good deals fast.

4. As a customer, I want to see the total bill before ordering, so that I know the exact price.

5. As a customer, I want to track my order status, so that I know when it will be ready.

6. As a manager, I want to update the menu myself, so that I can change prices anytime.

7. As a manager, I want to see all orders in one place, so that I don't need phone calls.

8. As a manager, I want to accept or reject reservations, so that I can manage tables properly.

3.Functional Requirements:

- **Authentication:**

01: The system shall let a user register as either a Customer or a Manager.

02: The system shall provide login access using a registered email and password.

03: The system shall check the email format and password strength during registration.

04: The system shall display an error message if login details are incorrect.

- **Menu:**

05: The system shall organize and display menu items under their categories.

06: The system shall mark special items with a "Daily Special" label.

07: The system shall keep the Add button disabled for items that are out of stock.

08: The system shall display a loading indicator until the menu data is ready.

- **Search:**

09: The system shall let the user search for menu items by typing a name.

10: The system shall display a "no results found" message for unmatched searches.

11: The system shall keep a record of the user's last five searches.

- **Cart:**

12: The system shall let the user add menu items to a shopping cart.

13: The system shall let the user update item quantity or remove it from the cart.

14: The system shall let the user enter a promo code to receive a discount.

15: The system shall automatically calculate the subtotal, tax, service charge, and final total.

- **Reservation:**

16: The system shall let the user choose a date, time slot, and number of guests.

17: The system shall mark time slots as unavailable when no table is free.

18: The system shall let the user cancel an existing reservation.

- **Orders:**

19: The system shall let the user place an order as either Dine-in or Takeaway.

20: The system shall move the order status forward automatically over time.

- **Manager Dashboard:**

21: The system shall let the manager view incoming orders and update their status.

22: The system shall let the manager add new items, edit prices, or remove items from the menu.

4.Non-Functional Requirements:

- **Usability**

The app shall have a simple and clear interface so that a new user can
place an order without any training.

- **Performance**

The menu screen shall load within 2 seconds, and scrolling through the
list shall remain smooth even with many items.

- **Responsiveness**

The app layout shall adjust properly on different screen sizes (small
phones to large tablets).

- **Maintainability**

The code shall be organized into folders (components, screens, context,

hooks) with reusable components so that future changes are easy to make.

- **Data Handling**
  All data (users, menu, orders, reservations) is stored in a MongoDB database and accessed through a Node.js and Express REST API. The app fetches and updates data from the server, so every device sees the same data and nothing is lost when the app is reinstalled.

5. Server-Side Data Model (MongoDB):

This app uses a MongoDB database. Data is stored in collections (users, menu items, orders, reservations) and the app reads and writes it through the server APIs.

- **Users**

Fields: id, name, email, password, role (Customer or Manager)

This stores the login details of customers and managers.

- **Categories**

Fields: id, name

This stores the menu categories, like Starters, Mains, Desserts, and Drinks.

- **MenuItems**

Fields: id, name, description, price, category, image, isSpecial, isAvailable

This stores all the food items shown on the menu.

- **Tables**

Fields: id, seats, isAvailable

This stores information about restaurant tables, used when booking a reservation.

- **Reservations**

Fields: id, userId, tableId, date, time, partySize, status

This stores table booking details made by customers.

- **Orders**

Fields: id, userId, items, total, type (Dine-in or Takeaway), status, timestamp

This stores order details, like what was ordered and its current status .

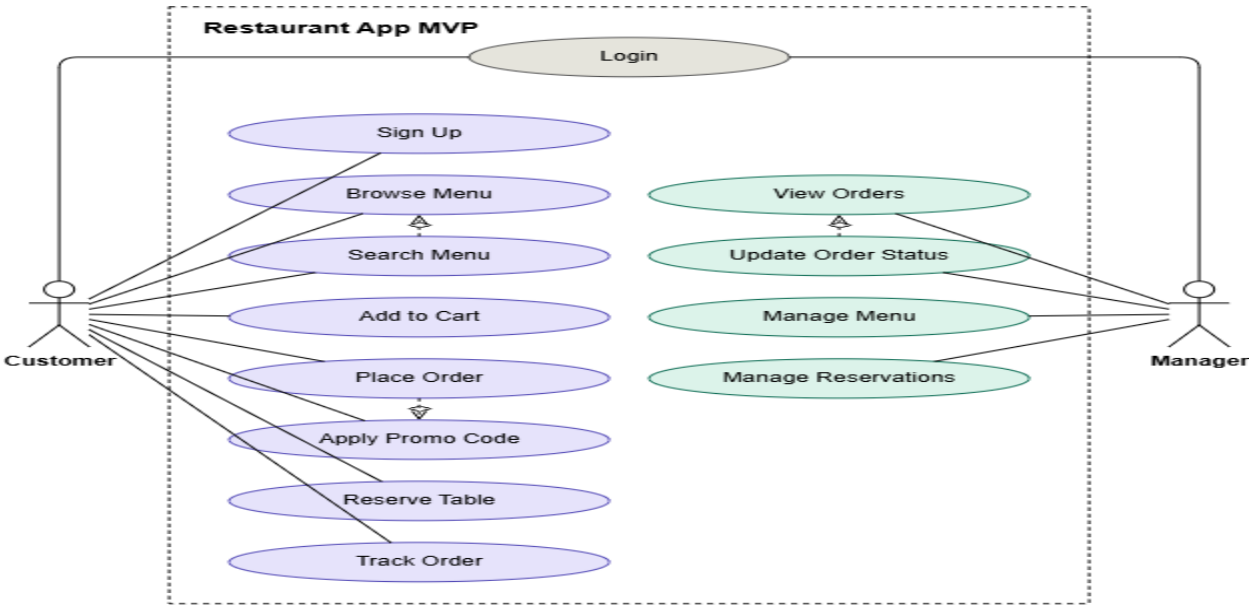
6.UML Diagram (Placeholders):

This section will contain the UML diagrams created in Question 2:

Use Case Diagram, Class Diagram, Sequence Diagram, State Machine Diagram,

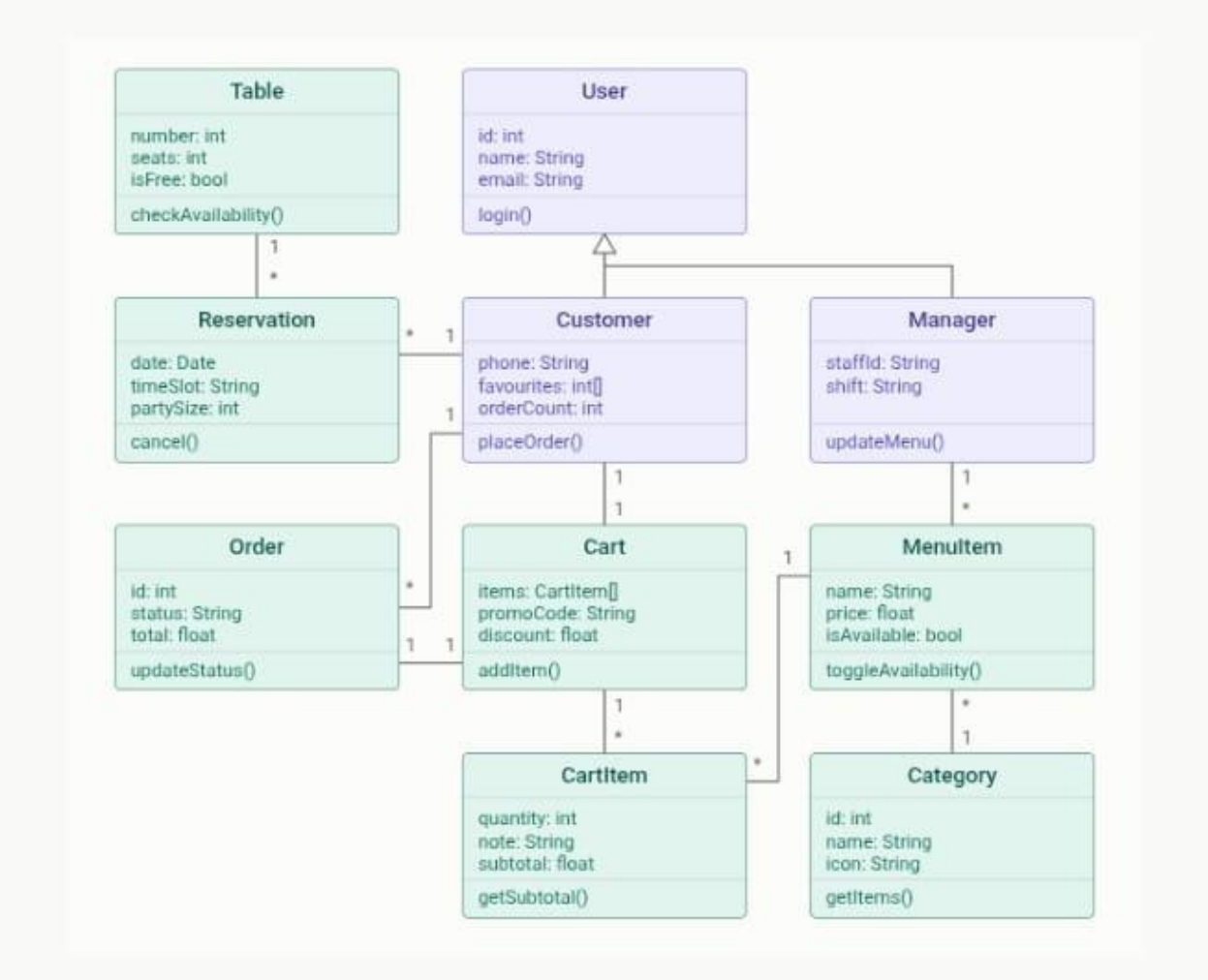
and Component Diagram. Each diagram will be inserted here with a caption

and a short paragraph explaining it.



Usecase diagram of the restaurant app mvp

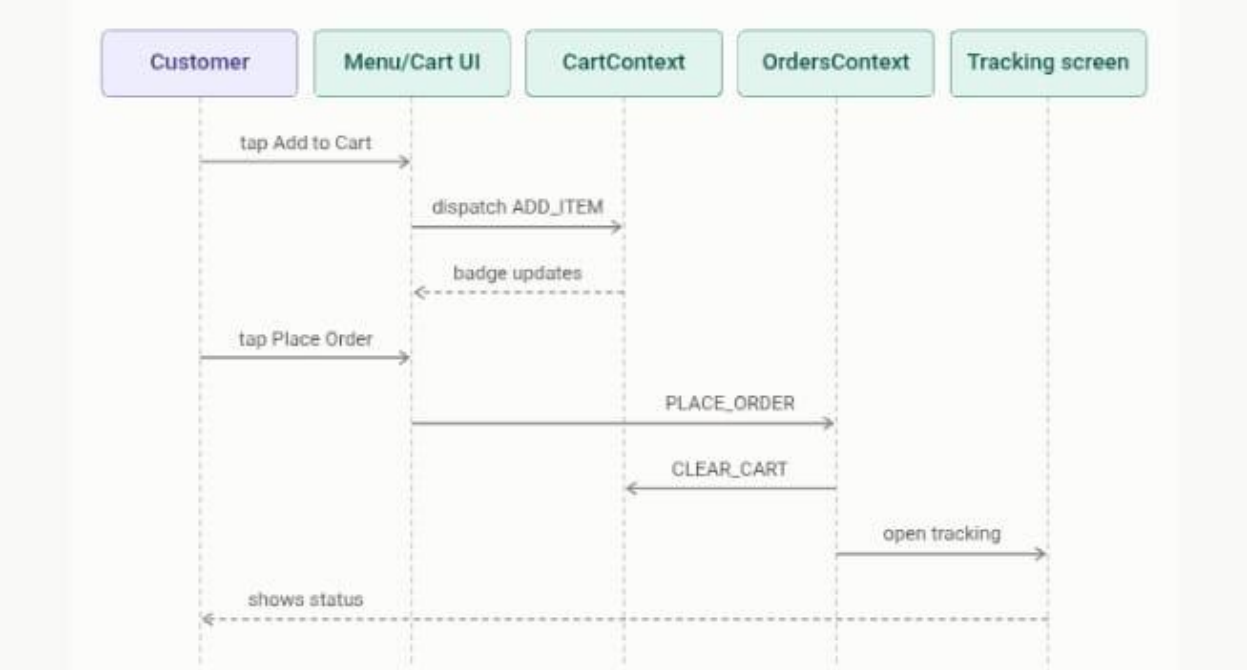
The use case diagram shows the Customer and the Restaurant Manager and what each can do in the app. The Customer can log in, sign up, browse and search the menu, add to cart, place an order, reserve a table, and track the order. The Manager can view orders, update order status, manage the menu, and manage reservations. Place Order includes Apply Promo Code.



Class diagram of the restaurant app mvp

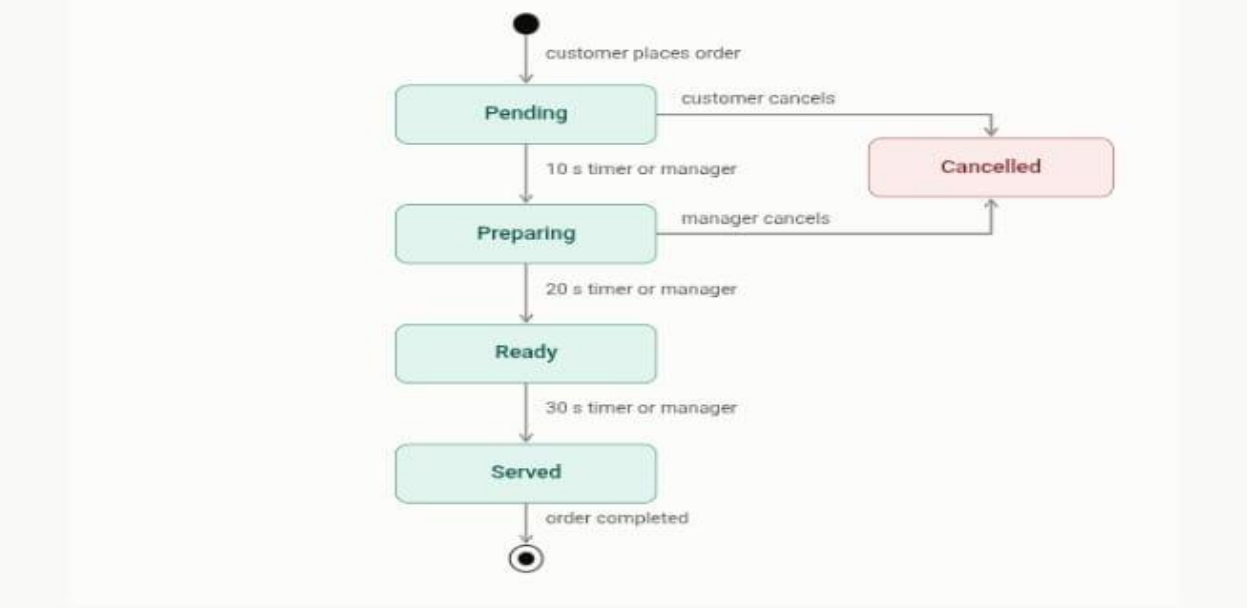
The class diagram shows the main classes of the app. Customer and Manager inherit from User. A Customer has a Cart, which contains CartItems linked to MenuItems, and MenuItems belong to a Category. A Customer can place Orders and make Reservations, and each Reservation is for a Table. The Manager manages the MenuItems.

iad



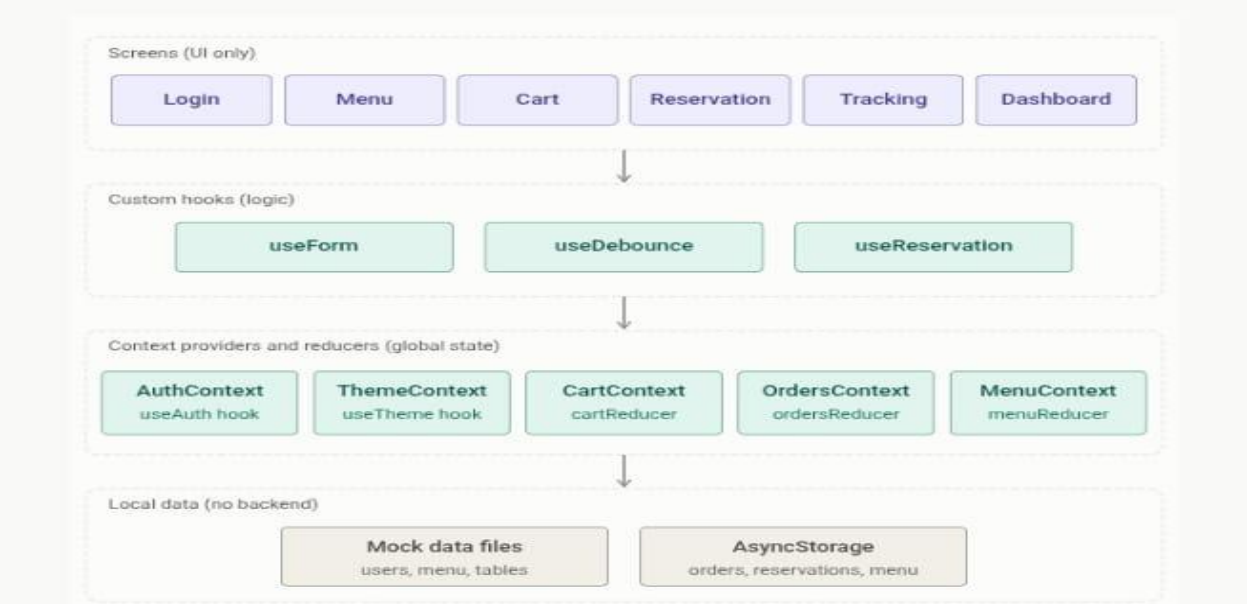
Sequence diagram of placing an order

The sequence diagram shows how a customer places an order. The customer taps Add to Cart, and the screen sends an ADD_ITEM action to the CartContext, which updates the cart badge. When the customer taps Place Order, a PLACE_ORDER action is sent to the OrdersContext, which clears the cart and opens the Order Tracking screen. The tracking screen then shows the order status to the customer.



State machine diagram of the order lifecycle

The state machine diagram shows the life of an order. A new order starts as Pending, then moves to Preparing, Ready, and finally Served. Each step happens automatically after a timer (10, 20, and 30 seconds in the demo) or when the manager changes the status. An order can be Cancelled by the customer while Pending, or by the manager while Pending or Preparing.



Component diagram of the react native app

The component diagram shows how the React Native app is organised in layers. The screens only show the UI and use custom hooks such as useForm, useDebounce, and useReservation for their logic. The hooks and screens use context providers (Auth, Theme, Cart, Orders, Menu), and the cart, orders, and menu contexts use reducers to manage state. The contexts fetch data from the server

through API calls (api.js) and send orders, reservations, and menu changes to the server, which saves them in MongoDB.

7. MVP Frontend Development (React Native):

1. Login and Signup Screen:

Purpose: Lets a user log in or create a new account as a Customer or Manager.

Hooks used: useState (to manage form inputs, errors, and loading state)

2. Menu Browsing Screen:

Purpose: Shows the restaurant's menu, organized by category, with loading

and error handling.

Hooks used: useState, useEffect (to load menu data and filter it)

3. Search and Scroll Controls:

Purpose: Lets the user search for menu items and scroll smoothly through

the list.

Hooks used: useRef (to manage the search input, debounce, and scroll position)

4. Profile and Theme Screen:

Purpose: Shows the logged-in user's details and lets them switch between

light and dark mode.

Hooks used: useContext (to access shared user and theme data)

5. Cart Screen

Purpose: Shows items added to the cart, lets the user update quantity,

remove items, and apply a promo code.

Hooks used: useReducer (to manage cart actions like add, remove, and promo)

6. Order Summary Screen

Purpose: Shows the final bill with subtotal, tax, service charge, and total.

Hooks used: useMemo, useCallback (to avoid unnecessary recalculations)

7. Table Reservation Screen

Purpose: Lets the user pick a date, time, and party size to book a table.

Hooks used: Custom hooks (to handle reservation logic separately from the UI)

8. Order Tracking and Manager Dashboard

Purpose: Shows the customer their order status live, and lets the manager

view/update orders, reservations, and the menu.

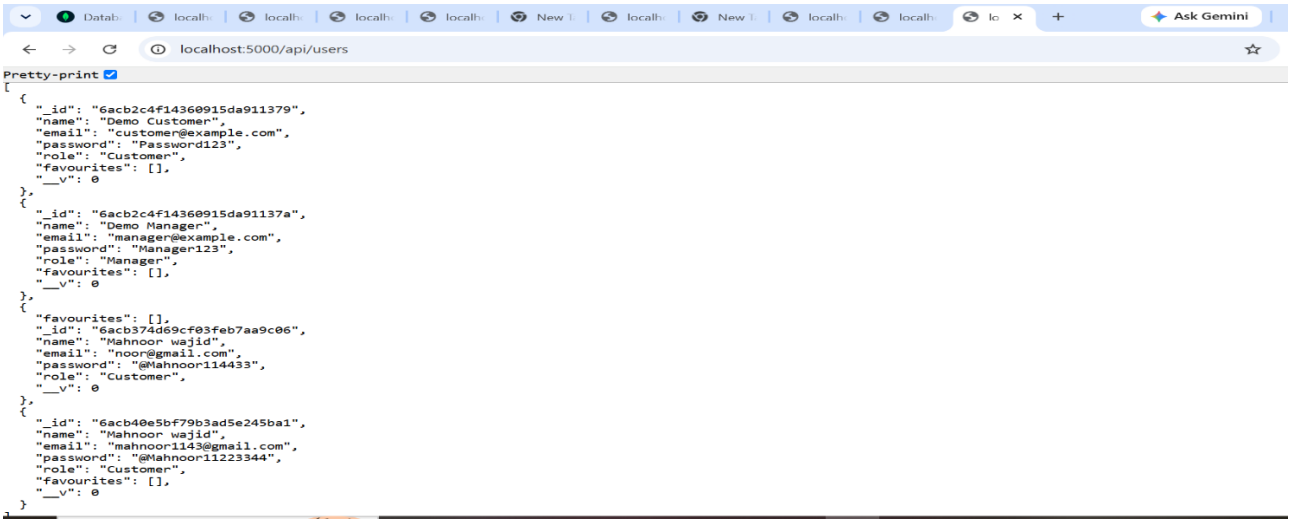
Hooks used: useEffect (for timers and saving data locally)

- **Note on Diagrams:**
  The diagrams in this document show the Assignment 1 design, where data came from mock files and AsyncStorage. In Assignment 2 this local data layer is replaced by a Node.js and Express server with a MongoDB database, and the app reads and writes data through api.js.
- **Problems in A1(Question 1)**
  While using the A1 app on different phones, the following problems were found:
  Data was not shared: a change on one phone was not visible on another phone.
  Manager changes (price or dish unavailable) were not seen by customers on their phones.
  Orders and reservations were lost when the app was reinstalled.
  Login accounts existed only on one phone and could not be used on another.
  An order placed on one phone was not visible to the manager on another phone.
- **System Architecture:**
  The system has three parts. The React Native app (client) sends requests to the Node.js and Express server through REST APIs. The server reads and writes data in the MongoDB database and sends the response back to the app. All phones use the same server, so all users see the same data.
  React Native App -> Express Server (REST API) -> MongoDB
  - **Testing Evidence:**

  Test 1: A customer placed an order on one phone, and the manager saw it on another phone.
  Test 2: The manager marked the order Ready and the customer saw the status change on their phone.
  Test 3: The manager marked Tomato Soup unavailable and the customer saw it unavailable.
  Test 4: A new account was registered and saved in MongoDB.

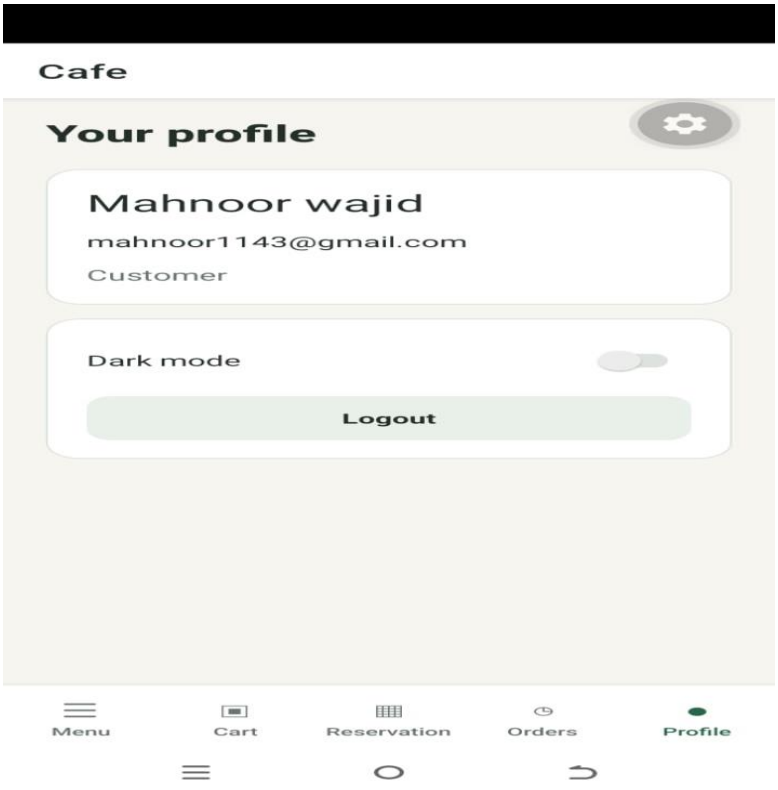
  - **SCREENSHOTS:**

- **Register_user_list:**

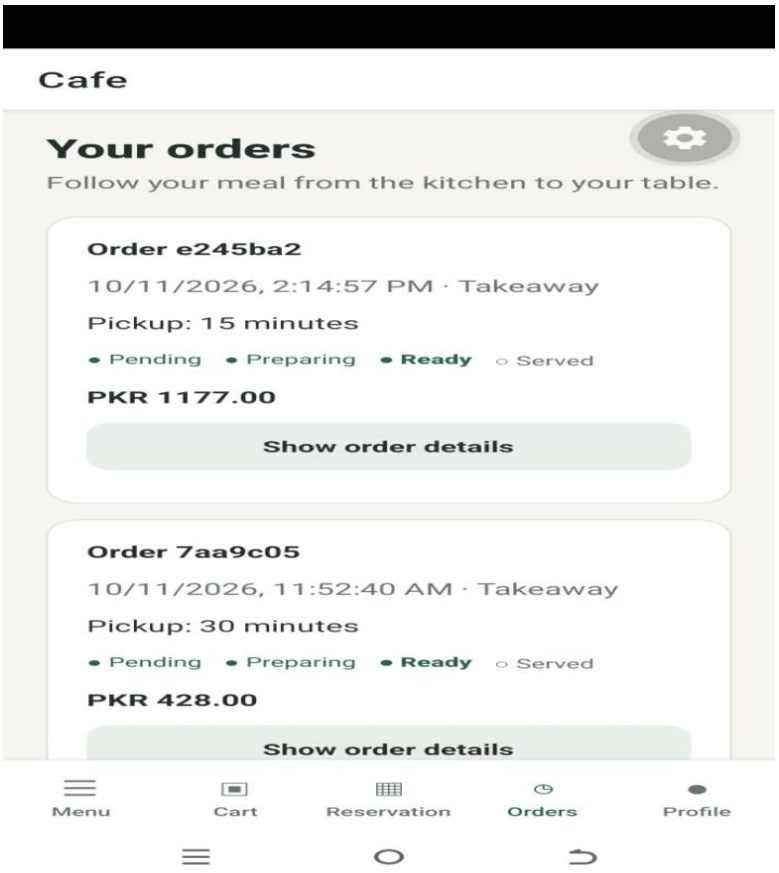


New accounts saved in MongoDB

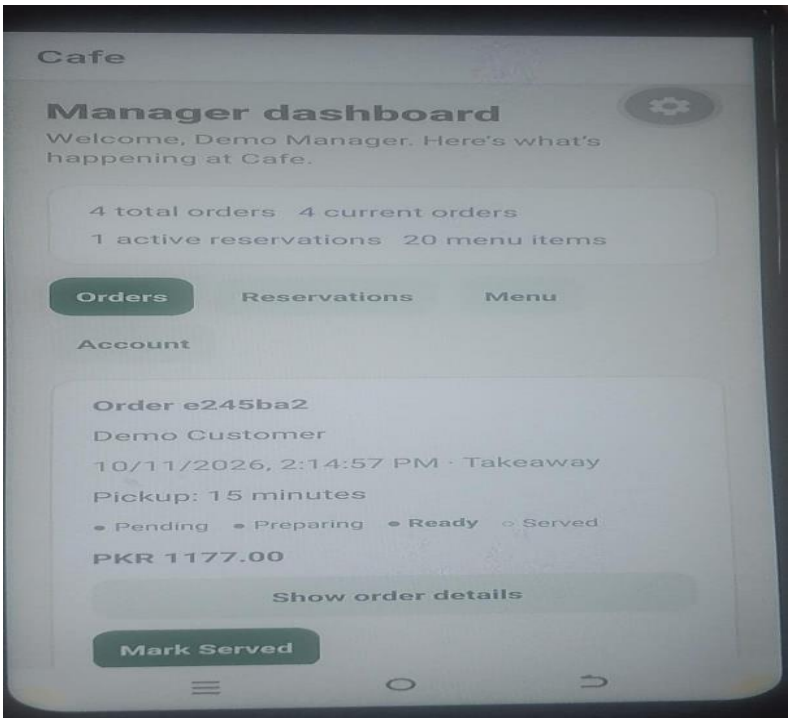
- **Profile_screen.png:**



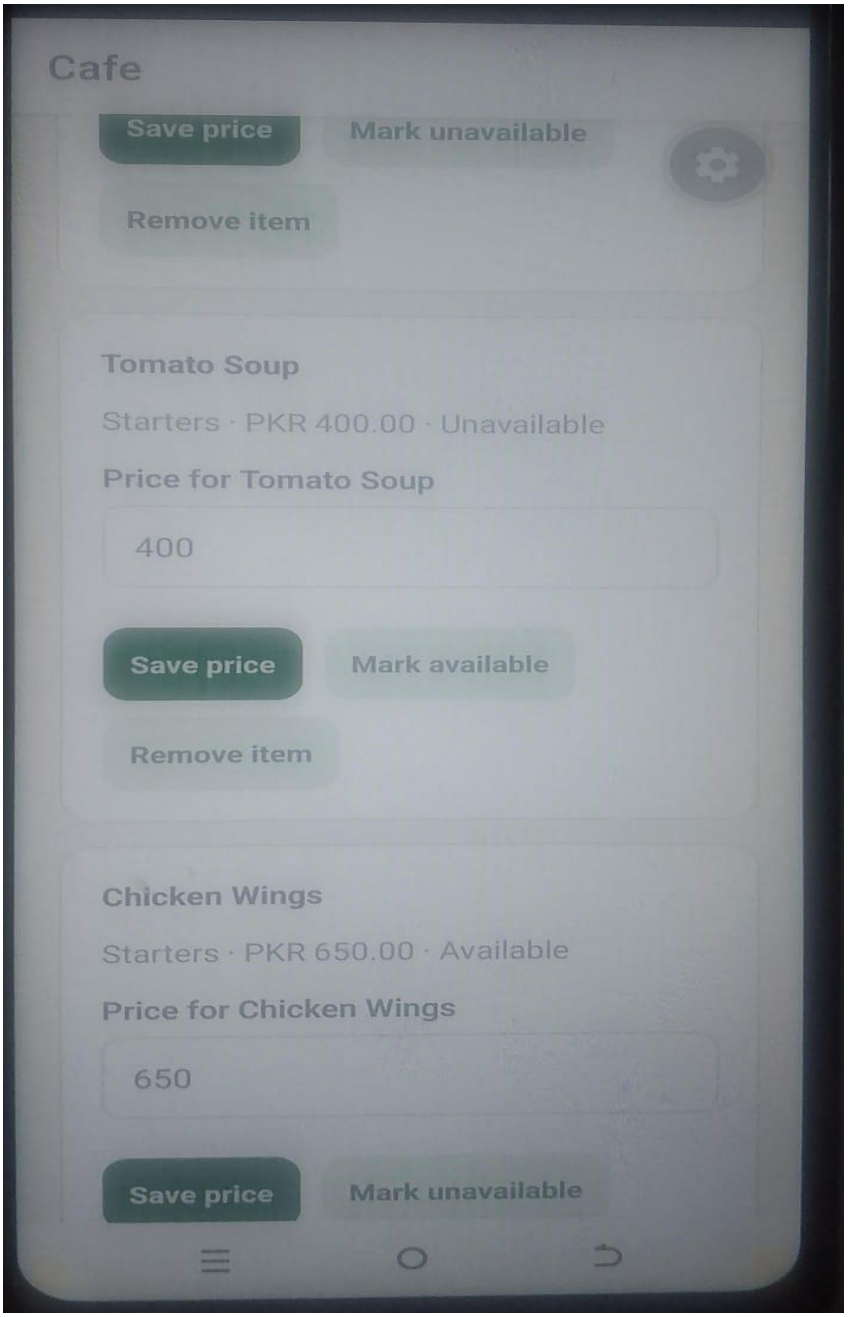
- **Order status ready on customer phone:**



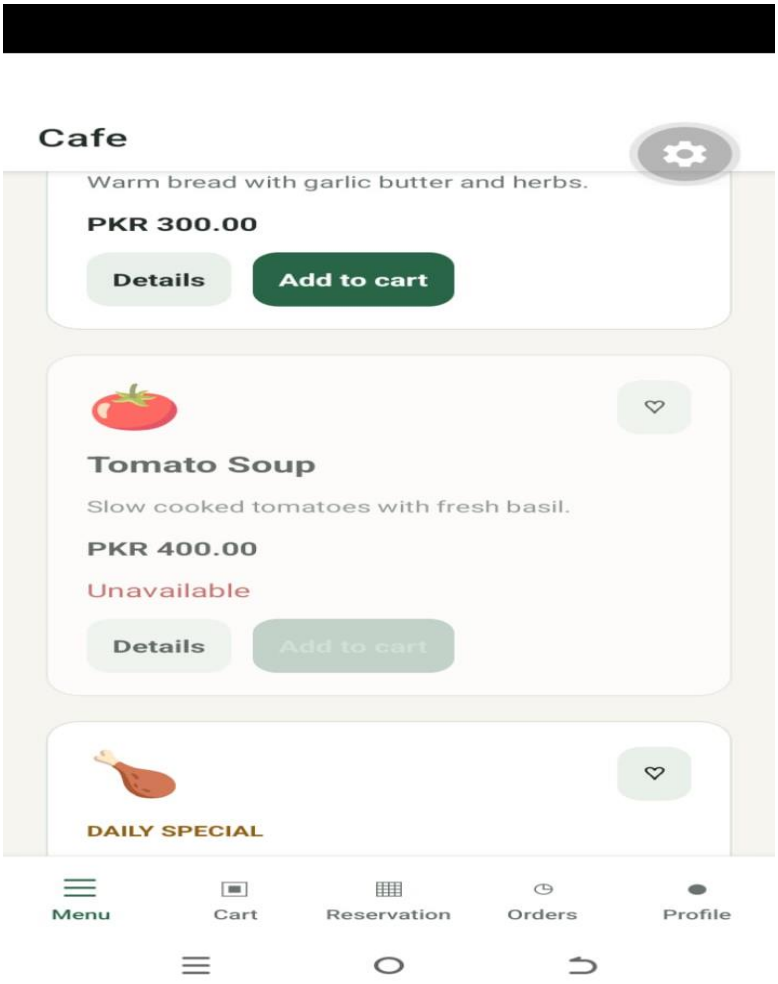
Manager marked the order ready:



Manager marks tomato soup unavailable :



Customer sees tomato soup unavailable:



API Endpoints:

Method	URL	Purpose
GET	/api/menu	Get all menu items
POST	/api/menu	Add a new menu item
PUT	/api/menu/:id	Update price or availability of a menu item
DELETE	/api/menu/:id	Remove a menu item
POST	/api/users/register	Create a new account
POST	/api/users/login	Log in a user
GET	/api/users	Get all users
GET	/api/reservations	Get reservations
POST	/api/reservations	Create a reservation
PUT	/api/reservations/:id	Update reservation status
DELETE	/api/reservations/:id	Cancel a reservation
GET	/api/orders	Get all orders
POST	/api/orders	Place a new order
PUT	/api/orders/:id	Update order status

Database Design(MongoDB):

The system uses MongoDB with four collections:

User: name, email (unique), password, role (default customer), favourites (list)

MenuItem: name, description, category, price, emoji, available

Reservation: userId, name, date, time, table, guests, phone, status

Order: userId, items, total, status, orderType, pickup time, createdAt